06강

프로그래밍 언어론

프로그래밍 언어의 구현

컴퓨터과학과 김진욱교수

학습목차



1. 프로그래밍 언어 정의와 구현
2. 프로그래밍 언어 구현 방법
3. 언어 구현 실제

01

프로그래밍 언어 정의와 구현

1. 프로그래밍 언어 정의와 구현

프로그래밍 언어 정의

- 구문 규칙 + 의미 규칙

구문 규칙

- 어떤 프로그램이 올바른 형태인지 규정하는 것

의미 규칙

- 올바른 형태의 프로그램을 실행하였을 때
  어떻게 실행되는 것이 올바른 것인지 규정하는 것

1. 프로그래밍 언어 정의와 구현

프로그래밍 언어 정의 방법

구문 규칙 정의 방법

- 문맥 자유 문법, BNF, EBNF, 구문 도표 등
- 문맥 자유 문법과 EBNF를 주로 사용

의미 규칙 정의 방법

- 기능적 의미론, 표기적 의미론, 공리적 의미론 등
- 자연어를 주로 사용

1. 프로그래밍 언어 정의와 구현

프로그래밍 언어 정의 예: 로봇 제어 언어

구문 규칙

```
<program> ::= { forward | left | right }
              ;
```

의미 규칙

- forward: 로봇이 향하고 있는 방향으로 단위 길이만큼 이동
- left: 로봇이 90도 좌회전
- right: 로봇이 90도 우회전

1. 프로그래밍 언어 정의와 구현

프로그래밍 언어 구현

- 그 프로그래밍 언어로 작성된 프로그램을 수행하는 프로그램
- 대개 프로그램을 입력으로 받는 프로그램 형태로 주어짐

프로그래밍 언어 L의 구현

- 다음을 수행하는 프로그램

1) 프로그래밍 언어 L로 작성된 프로그램 P_L이 L의 구문 규칙을 따르는 올바른 프로그램인지 검사
2) 올바른 경우, P_L을 입력으로 받아서 L의 의미 규칙에 따라 실행

1. 프로그래밍 언어 정의와 구현

프로그래밍 언어 구현

CPU의 함수 모형

$$M[\![P_M]\!](in) = out$$

- M: CPU가 받아들이는 기계어
- P_M: 기계어 M으로 작성된 프로그램
- in: 입력 데이터
- out: 출력 데이터

1. 프로그래밍 언어 정의와 구현

프로그래밍 언어 구현

프로그래밍 언어 구현의 함수 모형

$$L[\![P_L]\!](in) = out$$

- L: 프로그래밍 언어
- P_L: L로 작성된 프로그램
- in: 입력 데이터
- out: 출력 데이터

1. 프로그래밍 언어 정의와 구현

❑ 프로그래밍 언어 구현 형태

인터프리터의 함수 모형

$$Int_L[\![P_L]\!](in) = out$$

- Int_L : 프로그래밍 언어 L의 명령어를 해석하는 인터프리터
- 인터프리터는 L로 작성된 프로그램 P_L을 해석하며 CPU와 비슷한 기능을 수행함

1. 프로그래밍 언어 정의와 구현

❑ 프로그래밍 언어 구현 형태

컴파일러의 함수 모형

$$Comp_L[\![P_L]\!] = P_M$$

- $Comp_L$: 프로그래밍 언어 L의 컴파일러
- 고수준 언어 프로그램 P_L을 CPU가 수행할 수 있는 저수준 언어 프로그램 P_M으로 변환
- 이렇게 생성된 프로그램 P_M은 CPU가 수행할 수 있음

$$M[\![P_M]\!](in) = out$$

02

프로그래밍 언어 구현 방법

2. 프로그래밍 언어 구현 방법

프로그래밍 언어 구현 개요

- 전통적인 프로그래밍 언어 구현
  - 대상: 명령형 언어, 절차형 언어, 객체지향 언어
  - 기계어를 확장하는 형태로 구현

- 새로운 패러다임의 프로그래밍 언어 구현
  - 대상: 함수형 언어, 논리 언어
  - 구현 모델을 추상기계로 만들어 징검다리 삼아 구현

2. 프로그래밍 언어 구현 방법

전통적인 프로그래밍 언어 구현

명령형 언어

- 저급언어의 연산과 명령어를 확장하는 형태로 구현

절차형 언어

- 명령형 언어 + 사용자정의연산(함수)과 사용자정의 명령어 (프로시저)를 지원하는 형태로 구현

객체지향언어

- 절차형 언어 + 사용자정의 데이터 타입을 지원하는 형태로 구현

2. 프로그래밍 언어 구현 방법

새로운 패러다임의 프로그래밍 언어 구현

- 기계어를 확대한 형태로는 한계가 있음
- 언어의 계산 모델과 하드웨어 사이에 추상기계를 놓고,
  해당 언어의 프로그램을 추상기계가 이해하는 프로그램으로 변경

	함수형 언어	논리 언어
계산 모델	람다 계산법	연역 논리
추상기계	CPS, G-machine, SGM, STGM, TIM 등	WAM 등

- 가상기계: 추상기계가 구체적인 구현물로 제시되고
  코드를 독자적으로 수행할 수 있는 경우

2. 프로그래밍 언어 구현 방법

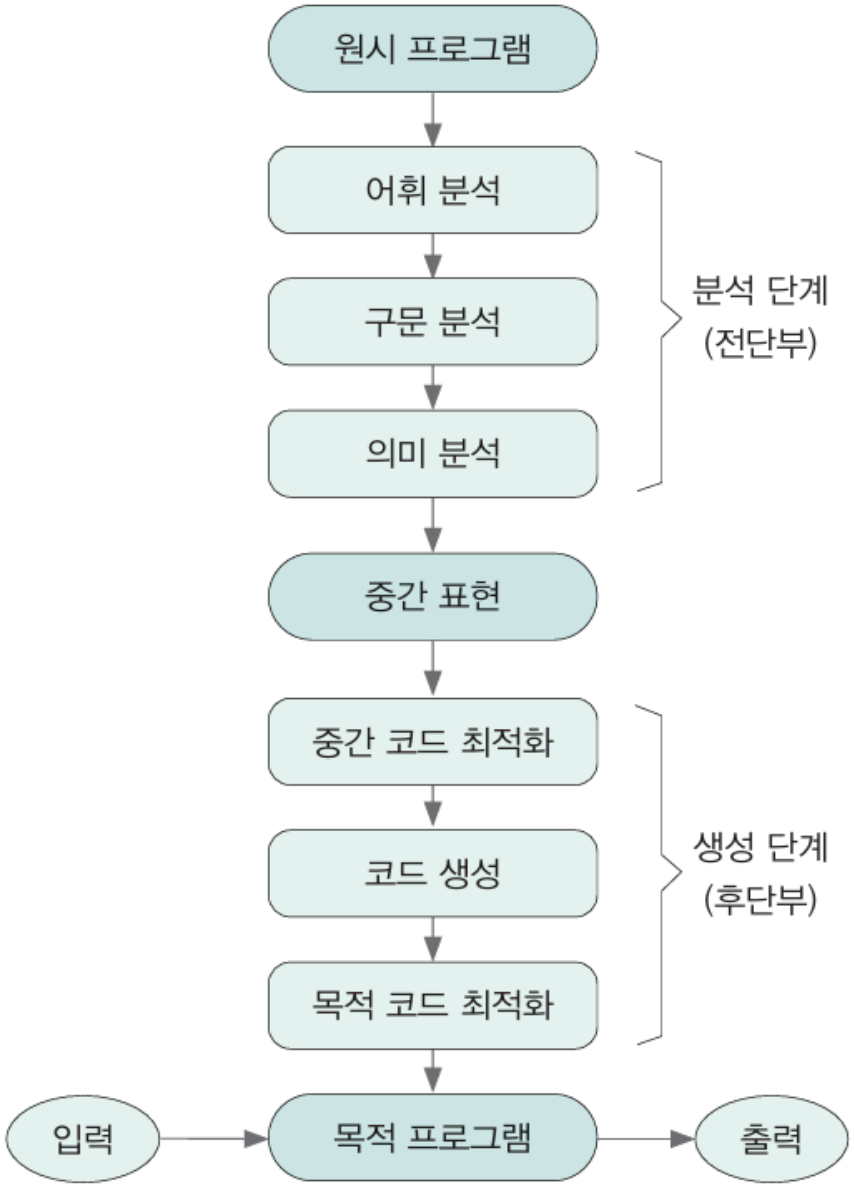
컴파일러 구현 단계

분석 단계(전단부)

- 주어진 프로그램의 구조 파악
- 중간 표현(중간 코드) 생성
- 프로그래밍 언어에 종속적

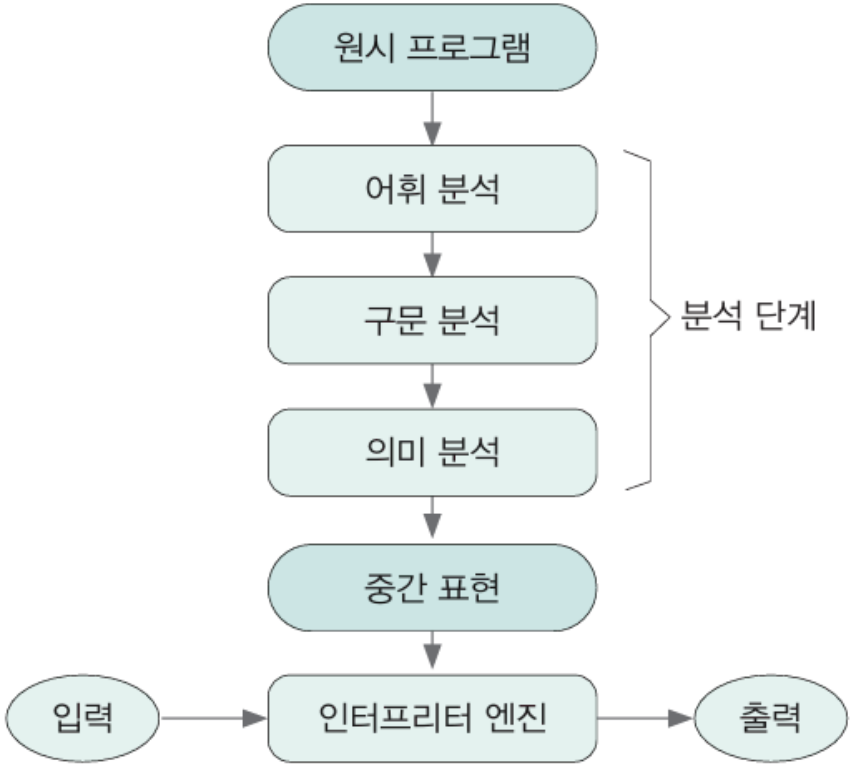
생성 단계(후단부)

- 목적 기계에 적합한 명령어 생성
- 효율적인 목적 코드 생성
- 목적 기계에 종속적



2. 프로그래밍 언어 구현 방법

- 인터프리터 구현 단계
  - 컴파일러 구현 단계의 분석 단계를 그대로 거침
  - 중간 표현을 순회하며 프로그램을 수행
  - 인터프리터 엔진
    - 프로그래밍 언어의 문장 단위로 해석하여 수행



2. 프로그래밍 언어 구현 방법

언어 구현에 필요한 자료 구조

구문 트리

- 언어 구현 단계의 중심을 차지하는 자료 구조
- 분석 단계의 전 과정을 관통

심볼 테이블 (컴파일러 구현)

- 프로그램에서 정의하거나 선언한 식별자 정보(타입, 선언 위치 등)를 저장

환경 (인터프리터 구현)

- 심볼 테이블보다 확장되어 식별자의 값 정보도 알 수 있는 테이블

2. 프로그래밍 언어 구현 방법

언어 구현에 필요한 자료 구조

- 실행 환경
  - 언어를 구현할 때 사용되지는 않지만 프로그램 실행 시 반드시 필요

메모리 구조: 실행 지원

- 정적 세그먼트: 코드, 정적 데이터
- 동적 세그먼트: 스택, 힙

레지스터: 메모리 및 실행 상태를 관리

- 전용 레지스터: PC, SP, FP 등
- 범용 레지스터: 기타 여러 목적으로 사용되는 레지스터

03

언어 구현 실제

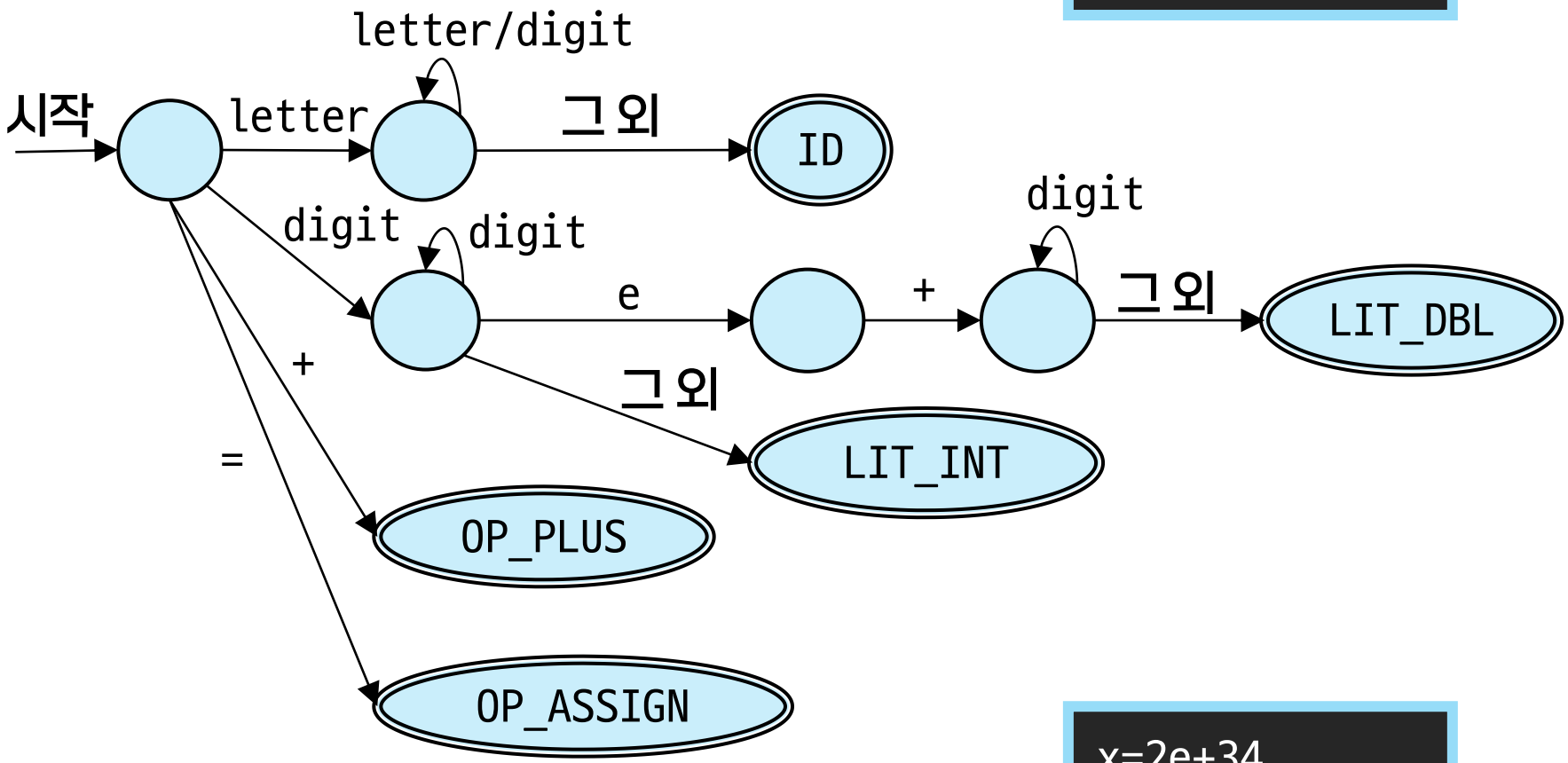
3. 언어 구현 실제

- 어휘 분석기 구현
  - 어휘 분석기는 프로그램의 어휘를 구별해 냄
    - 어휘: 예약어, 리터럴, 연산자 등
  - 필요에 따라 속성을 구하여 구문 분석기에 전달
  - 대부분 유한 상태 기계(FSM)를 구성하여 구현

3. 언어 구현 실제

어휘 분석기 구현 예

x=2+34

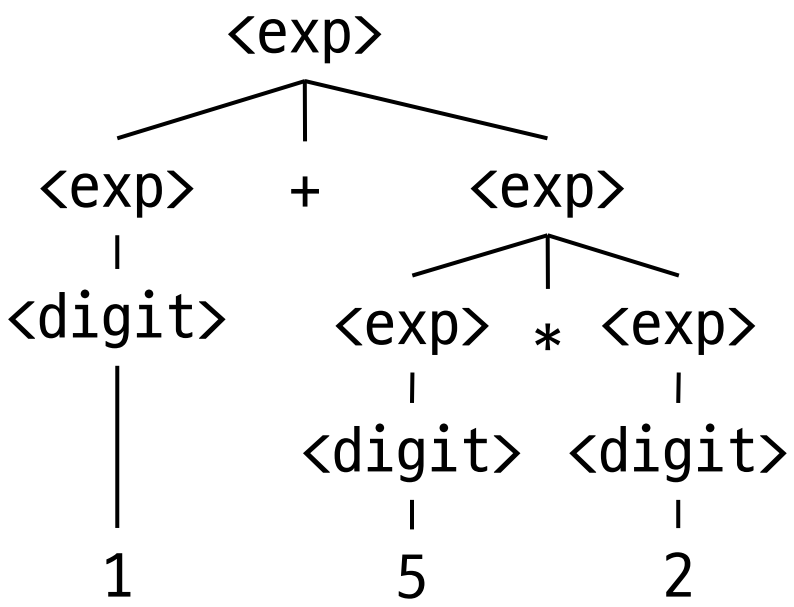


x=2e+34

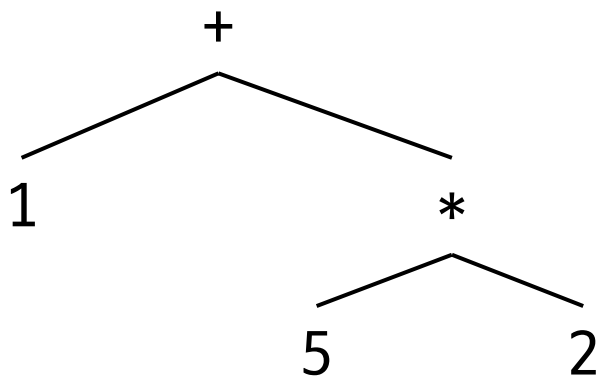
3. 언어 구현 실제

구문 분석기 구현

- 구문 분석기는 어휘 분석기의 분석 결과인 토큰 열로부터 구문 트리를 구성함
- 구문 트리의 형태: 파스 트리, 추상 구문 트리
  - 파스 트리: 문법 기호 정보를 모두 포함



- 추상 구문 트리(AST): 번역에 필요한 정보만 포함



3. 언어 구현 실제

구문 분석기 구현

- 구문 트리를 순회하는 여러 프로시저로 구성된 프로그램

순환 하강 구문 분석기

- 문법 규칙을 그대로 코드로 바꾼 형태

1) 각 비단말 기호의 문법 규칙에 대해
하나의 프로시저를 만들되,
우변을 모사하도록 프로시저를 만듦

2) 우변을 모사할 때
단말 기호라면 일치하는지 검사하고,
비단말 기호라면 해당 프로시저를 호출함

3. 언어 구현 실제

순환 하강 구문 분석기 예

- 괄호가 맞는 문자열을 생성하기 위한 문법

```
<L> ::= <S> newline <L>
      | EOF
      ;
<S> ::= '(' <S> ')' <S>
      | ε
      ;
```

```
()
()()()
()(())
```

```
())
)()(
(()
```

```c
#include <stdio.h>
#include <stdlib.h>

int LA = 0;
int nexttoken() {
  return getchar();
}

void error(char c) {
  printf("Syntax error near '%c'.\n", c);
  exit(-1);
}

void match(int token) {
  if (LA == token)
    LA = nexttoken();
  else
    error(token);
}

void L(), S();
```

```c
void L() {
  if (LA == EOF)
    ;
  else {
    S(); match('\n'); L();
  }
}

void S() {
  if (LA == '(') {
    match('('); S(); match(')'); S();
  }
  else
    ;
}

int main() {
  LA = nexttoken();
  L();
  return 0;
}
```

```
<L> ::= <S> newline <L>
      | EOF
      ;
```

```
<S> ::= ‘(’ <S> ‘)’ <S>
      | ε
      ;
```

```
()
()()()
()(())
^Z
```

정리하기

- 프로그래밍 언어를 구현하기 위해서는 프로그램의 형태를 정의하는 구문론과 프로그램의 수행 의미를 정의하는 의미론으로 언어를 정의해야 함
- 전통적인 프로그래밍 언어는 기계의 단순한 명령어를 확대하는 형태로 구현하고, 새로운 패러다임의 언어는 구현 모델을 추상기계로 만들고 이를 정검다리로 삼아 구현하는 형태임
- 컴파일러의 구현 단계는 프로그래밍 언어에 의존적인 분석 단계 (전단부)와 목적 기계에 의존적인 생성 단계(후단부)로 나눌 수 있음
- 인터프리터의 구현도 분석 단계는 그대로 거치지만, 중간 표현 을 목적 코드로 바꾸는 대신 순회하며 프로그램을 실행함
- 구문 트리와 중간 표현 외에 프로그래밍 언어 구현에 필요한 자료 구조로는 심볼 테이블과 환경이 있음. 실행 환경도 고려해야 함

07강

다음시간안내 ▶▶▶

변수와바인딩